ablation

October 6, 2026

1 Ablación de nodo — bloques y cabezas (2026-08-22)

Objetivo: bloque B del capítulo (circuit study), punto **B.4** de `CLAUDE.md`: *"apagar (o neutralizar) un head a la vez y medir el impacto en el desempeño — importancia causal individual"*, con la referencia de Michel, Levy & Neubig (2019) *"Are Sixteen Heads Really Better than One?"* y Voita et al. (2019) *"Analyzing Multi-Head Self-Attention"*.

No adivinamos qué bloque o cabeza "es mejor" apagar — los apagamos todos, uno a la vez, y medimos la caída de accuracy. Eso es literalmente lo que hace un experimento de ablación causal: en vez de intuir cuál capa importa más, se mide.

Dos niveles de granularidad: 1. **Ablación de bloque completo**: el bloque i se salta por completo (ni atención ni MLP) — el residual stream pasa sin tocar. Es la versión más gruesa ("layer drop"). 2. **Ablación de cabeza individual**: dentro de un bloque, se apaga (pone en cero) la salida de una sola cabeza de atención antes de concatenar y proyectar — las demás cabezas de ese bloque y el resto del modelo siguen normales. Es la versión fina que pide B.4.

Con 4 bloques × 4 cabezas = 16 combinaciones, es barato correr todo el barrido.

Corre sobre los dos checkpoints entrenados en AwA2 (`vit-with-15-epochs-AwA2` y `vit-with-15-epochs-interpretable-AwA2`) en una sola pasada, y cada gráfica se genera y guarda dos veces (una por modelo). La hipótesis de B.4 es que el baseline tiene cabezas prescindibles y otras críticas, mientras que el modelo penalizado reparte la importancia de forma más uniforme entre cabezas — comparar los dos heatmaps guardados (sección 5) es la comparación directa.

[8]:
```
import json
import sys
from pathlib import Path

# Misma convención que el resto de notebooks/: vive 2 niveles bajo la raíz del␣
↪proyecto.
project_root = Path.cwd().resolve().parents[1]
print(f"Project root: {project_root}")
sys.path.append(str(project_root))

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F
```

```
from modules.model import ViTForClassfication
from modules.datasets import AwA2Dataset
from modules.figures import save_fig

torch.manual_seed(0)

EXPERIMENTS_DIR = project_root / "experimentation" / "experiments"
OUTPUT_DIR = EXPERIMENTS_DIR / "22082026_ablation"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Los dos modelos a comparar en todo el notebook (mismo config, misma seed de␣
 ↪datos).
MODELS = {
    "baseline": "vit-with-15-epochs-AwA2",
    "interpretable": "vit-with-15-epochs-interpretable-AwA2",
}
```

```
Project root: /Users/valefeve/Projects/interpretability of neural
networks/interpretable-transformers
```

1.1 1. Cargar los dos modelos

`map_location="cpu"` porque los checkpoints se guardaron desde MPS. La ablación por cabeza que armamos abajo asume `use_faster_attention=False` (cabezas como submódulos separados, `block.attention.heads[i]`) — lo verificamos con un `assert` para no fallar en silencio si algún día cambia.

[9]:
```
def load_model(name):
    exp_dir = EXPERIMENTS_DIR / name
    with open(exp_dir / "config.json") as f:
        config = json.load(f)
    with open(exp_dir / "metrics.json") as f:
        metrics = json.load(f)
    assert not config["use_faster_attention"], (
        "Este notebook asume block.attention.heads (cabezas separadas); "
        "con use_faster_attention=True habria que adaptar la ablacion por␣
 ↪cabeza."
    )
    model = ViTForClassfication(config)
    model.load_state_dict(torch.load(exp_dir / "model_final.pt",␣
 ↪map_location="cpu"))
    model.eval()
    return model, config, metrics


models = {key: load_model(name) for key, name in MODELS.items()}

config = models["baseline"][1]
```

```
assert config == models["interpretable"][1], "Los dos modelos deben tener la misma arquitectura"
n_blocks = len(models["baseline"][0].encoder.blocks)
n_heads = config["num_attention_heads"]

for key, (_, _, metrics) in models.items():
    print(f"{key:>13} ({MODELS[key]}): accuracy de entrenamiento = {metrics['accuracy']:.4f}")
print(f"Bloques: {n_blocks}  |  cabezas por bloque: {n_heads}")
```

```
     baseline (vit-with-15-epochs-AwA2): accuracy de entrenamiento = 0.3091
interpretable (vit-with-15-epochs-interpretable-AwA2): accuracy de entrenamiento
= 0.3143
Bloques: 4  |  cabezas por bloque: 4
```

1.2 2. Datos de test (AwA2)

`AwA2Dataset` usa el cache local ya construido en `data/awa2/` (ver `modules.datasets.prepare_awa2_cache`), así que no vuelve a descargar nada.

[10]:
```
dataset = AwA2Dataset(image_size=32, root=str(project_root / "data" / "awa2"))
_, testloader, classes = dataset.get_loaders(batch_size=256)
num_classes = dataset.n_classes

N_EVAL = 1000  # tamano del lote de evaluacion (CPU-friendly)
eval_images, eval_labels = dataset.get_samples_from_indices(range(N_EVAL), device="cpu", set="test")
print(f"Lote de evaluacion: {N_EVAL} imagenes de test, {num_classes} clases")
```

```
Lote de evaluacion: 1000 imagenes de test, 50 clases
```

1.3 3. Funciones de ablación

`forward_with_block_skipped`: reimplementa el forward manualmente, saltando por completo un bloque cuando `skip_block_idx` coincide (el residual stream no se toca ese paso).

`forward_with_head_ablated`: igual, pero en el bloque `ablate_layer_idx` reconstruye la atención a mano — corre cada cabeza por separado (como hace `MultiHeadAttention.forward` internamente) y pone en cero la salida de `ablate_head_idx` antes de concatenar y proyectar. El resto del bloque (las otras cabezas, el MLP) sigue intacto.

Con `skip_block_idx=None` / `ablate_layer_idx=None` ninguna de las dos toca nada — deben reproducir exactamente el forward normal del modelo. Lo verificamos abajo antes de confiar en el resto del notebook.

[11]:
```
@torch.no_grad()
def forward_with_block_skipped(model, x, skip_block_idx=None):
    h = model.embedding(x)
    for i, block in enumerate(model.encoder.blocks):
        if i == skip_block_idx:
```

```
            continue  # bloque completo salteado: el residual stream pasa sin␣
 ↪tocar
        h, _ = block(h, output_attentions=False)
    return model.classifier(h[:, 0, :])


@torch.no_grad()
def forward_with_head_ablated(model, x, ablate_layer_idx=None,␣
 ↪ablate_head_idx=None):
    h = model.embedding(x)
    for i, block in enumerate(model.encoder.blocks):
        if i == ablate_layer_idx:
            attn_in = block.layernorm_1(h)
            head_outputs = []
            for hidx, head in enumerate(block.attention.heads):
                out, _ = head(attn_in)
                if hidx == ablate_head_idx:
                    out = torch.zeros_like(out)
                head_outputs.append(out)
            concat = torch.cat(head_outputs, dim=-1)
            attn_out = block.attention.output_projection(concat)
            attn_out = block.attention.output_dropout(attn_out)
            h = h + attn_out
            mlp_out = block.mlp(block.layernorm_2(h))
            h = h + mlp_out
        else:
            h, _ = block(h, output_attentions=False)
    return model.classifier(h[:, 0, :])


@torch.no_grad()
def accuracy_of(logits, labels):
    return (logits.argmax(dim=-1) == labels).float().mean().item()
```

[12]:
```
# Sanity check: sin ablacion, ambas funciones deben coincidir EXACTAMENTE
# con el forward normal de cada modelo. También calculamos aquí␣
 ↪baseline_accuracy
# por modelo, reutilizada en las secciones 4 y 5.
baseline_accuracy = {}
for key, (model, _, _) in models.items():
    with torch.no_grad():
        direct_logits, _ = model(eval_images, output_attentions=False)

    block_fn_logits = forward_with_block_skipped(model, eval_images,␣
 ↪skip_block_idx=None)
    head_fn_logits = forward_with_head_ablated(model, eval_images,␣
 ↪ablate_layer_idx=None, ablate_head_idx=None)
```

```
    diff_block = (direct_logits - block_fn_logits).abs().max().item()
    diff_head = (direct_logits - head_fn_logits).abs().max().item()
    assert diff_block < 1e-5 and diff_head < 1e-5, f"{key}: las␣
 ↪reimplementaciones deben reproducir el forward normal"

    baseline_accuracy[key] = accuracy_of(direct_logits, eval_labels)
    print(f"{key:>13}: diff block={diff_block:.2e}  diff head={diff_head:.2e}  "
          f"accuracy baseline (n={N_EVAL}) = {baseline_accuracy[key]:.3f}")
print("OK - las reimplementaciones manuales son fieles al forward original, en␣
 ↪ambos modelos.")
```

```
    baseline: diff block=0.00e+00  diff head=0.00e+00  accuracy baseline
(n=1000) = 0.226
interpretable: diff block=0.00e+00  diff head=0.00e+00  accuracy baseline
(n=1000) = 0.230
OK - las reimplementaciones manuales son fieles al forward original, en ambos
modelos.
```

1.4 4. Ablación de bloque completo

Un bloque a la vez, saltado por completo. Si la accuracy cae poco al saltar un bloque, ese bloque es prescindible (su función la puede estar cubriendo el resto de la red); si cae mucho, es un bloque crítico.

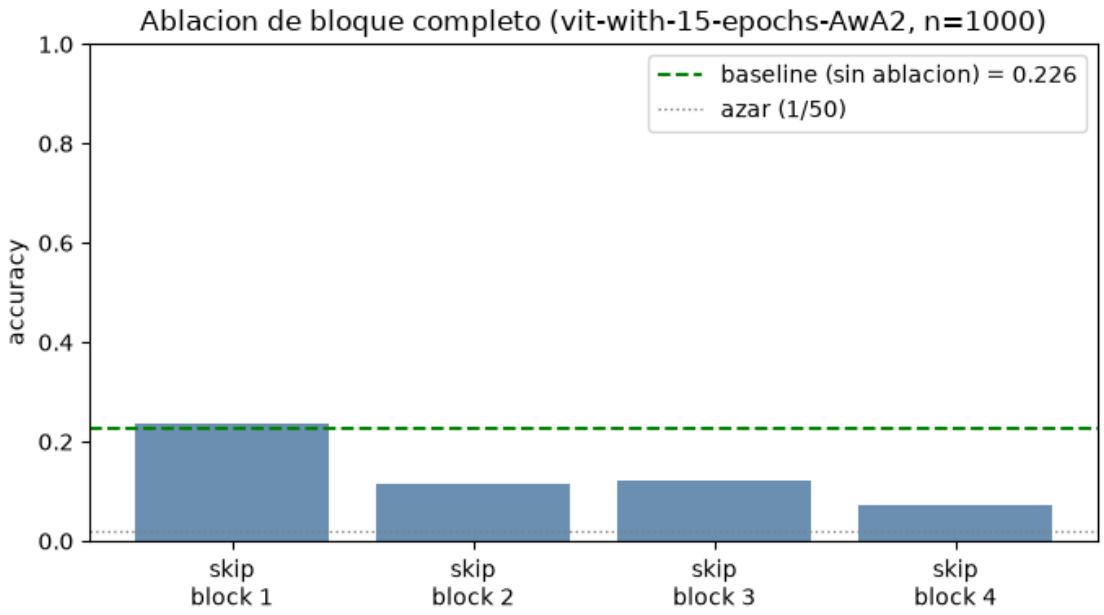
[13]:
```
for key, (model, _, _) in models.items():
    block_accuracies = []
    for i in range(n_blocks):
        logits = forward_with_block_skipped(model, eval_images,␣
 ↪skip_block_idx=i)
        acc = accuracy_of(logits, eval_labels)
        block_accuracies.append(acc)
        print(f"[{key}] skip block {i+1}: accuracy = {acc:.3f}  "
              f"(caida vs baseline: {baseline_accuracy[key] - acc:+.3f})")

    fig, ax = plt.subplots(figsize=(7, 4))
    labels_x = [f"skip\nblock {i+1}" for i in range(n_blocks)]
    ax.bar(labels_x, block_accuracies, color="#6b8fb0")
    ax.axhline(baseline_accuracy[key], color="green", linestyle="--",␣
 ↪linewidth=1.5,
               label=f"baseline (sin ablacion) = {baseline_accuracy[key]:.3f}")
    ax.axhline(1 / num_classes, color="gray", linestyle=":", linewidth=1,␣
 ↪label=f"azar (1/{num_classes})")
    ax.set_ylabel("accuracy")
    ax.set_title(f"Ablacion de bloque completo ({MODELS[key]}, n={N_EVAL})")
    ax.set_ylim(0, 1)
    ax.legend()
    plt.tight_layout()
```

```
    save_fig(fig, "ablation_blocks", OUTPUT_DIR, tag=MODELS[key],␣
↪project_root=project_root)
    plt.show()

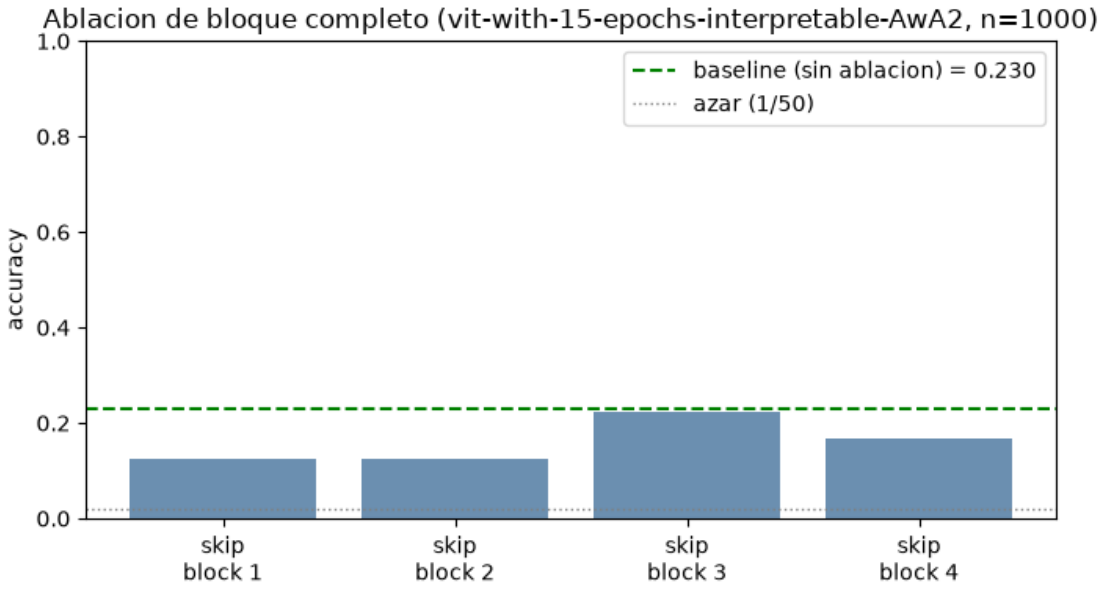
    most_critical_block = int(np.argmin(block_accuracies))
    print(f"[{key}] Bloque mas critico (mayor caida al saltarlo): block␣
↪{most_critical_block + 1}\n")
```

```
[baseline] skip block 1: accuracy = 0.238  (caida vs baseline: -0.012)
[baseline] skip block 2: accuracy = 0.116  (caida vs baseline: +0.110)
[baseline] skip block 3: accuracy = 0.122  (caida vs baseline: +0.104)
[baseline] skip block 4: accuracy = 0.072  (caida vs baseline: +0.154)
guardado: experimentation/experiments/22082026_ablation/ablation_blocks_vit-
with-15-epochs-AwA2.png
```




```
[baseline] Bloque mas critico (mayor caida al saltarlo): block 4

[interpretable] skip block 1: accuracy = 0.124  (caida vs baseline: +0.106)
[interpretable] skip block 2: accuracy = 0.125  (caida vs baseline: +0.105)
[interpretable] skip block 3: accuracy = 0.223  (caida vs baseline: +0.007)
[interpretable] skip block 4: accuracy = 0.168  (caida vs baseline: +0.062)
guardado: experimentation/experiments/22082026_ablation/ablation_blocks_vit-
with-15-epochs-interpretable-AwA2.png
```




```
[interpretable] Bloque mas critico (mayor caida al saltarlo): block 1
```

1.5 5. Ablación de cabeza individual

Las 4×4 combinaciones de (bloque, cabeza), cada una apagada por separado. El resultado clásico de este tipo de análisis (Michel et al. 2019, Voita et al. 2019) es que unas pocas cabezas concentran casi toda la importancia y muchas otras son casi redundantes — eso es justo lo que la hipótesis de B.4 quiere contrastar entre baseline y penalizado.

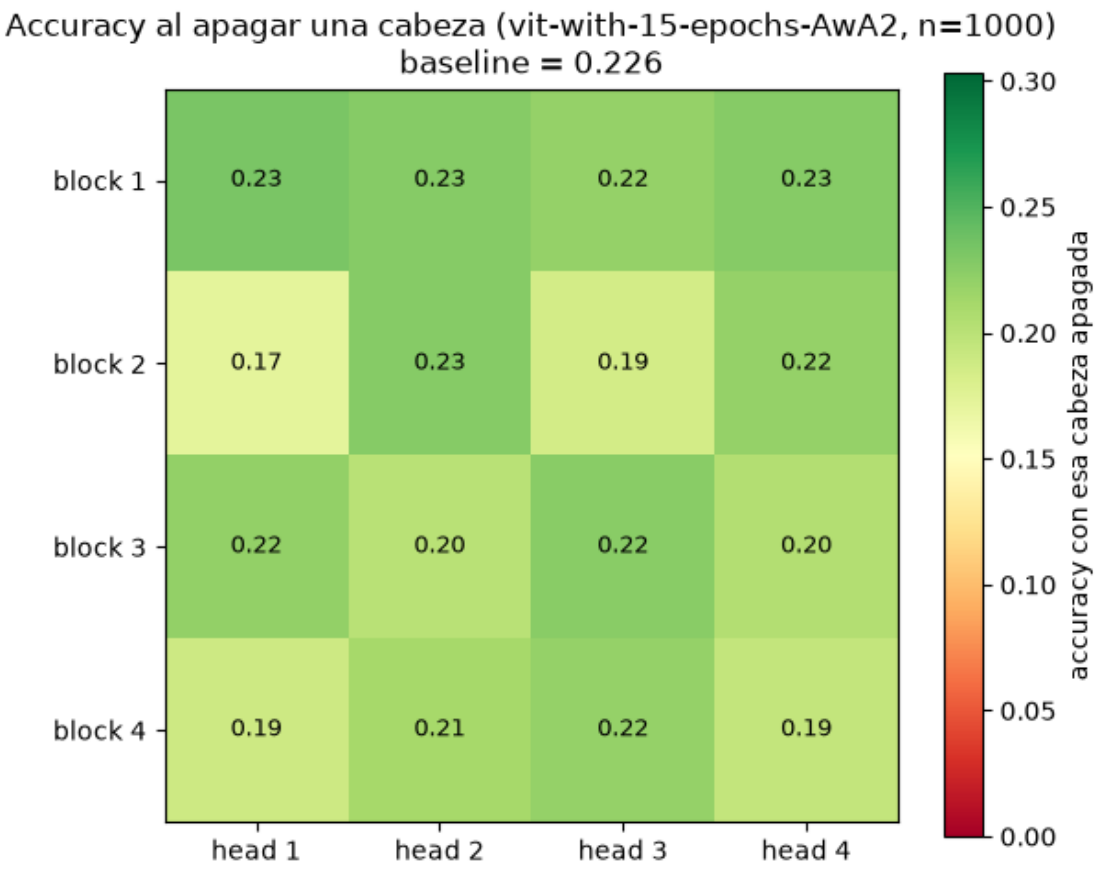
[14]:
```
head_accuracy = {}
for key, (model, _, _) in models.items():
    acc = np.zeros((n_blocks, n_heads))
    for layer_idx in range(n_blocks):
        for head_idx in range(n_heads):
            logits = forward_with_head_ablated(model, eval_images,␣
 ↪ablate_layer_idx=layer_idx, ablate_head_idx=head_idx)
            acc[layer_idx, head_idx] = accuracy_of(logits, eval_labels)
    head_accuracy[key] = acc

# Escala de color COMPARTIDA entre modelos: 30% arriba del máximo del baseline␣
 ↪(sin
# penalización). Con vmax por-modelo, el mismo color no significa el mismo␣
 ↪valor entre
# las dos figuras - con un vmax compartido sí es comparable a simple vista.
vmax_shared = head_accuracy["baseline"].max() * 1.3
```

```
for key in MODELS:
    fig, ax = plt.subplots(figsize=(6, 5))
    im = ax.imshow(head_accuracy[key], cmap="RdYlGn", vmin=0, vmax=vmax_shared)
    ax.set_xticks(range(n_heads))
    ax.set_xticklabels([f"head {h+1}" for h in range(n_heads)])
    ax.set_yticks(range(n_blocks))
    ax.set_yticklabels([f"block {l+1}" for l in range(n_blocks)])
    for l in range(n_blocks):
        for h in range(n_heads):
            ax.text(h, l, f"{head_accuracy[key][l, h]:.2f}", ha="center",␣
↪va="center", fontsize=9)
    ax.set_title(f"Accuracy al apagar una cabeza ({MODELS[key]},␣
↪n={N_EVAL})\nbaseline = {baseline_accuracy[key]:.3f}")
    fig.colorbar(im, ax=ax, label="accuracy con esa cabeza apagada")
    plt.tight_layout()
    save_fig(fig, "ablation_heads", OUTPUT_DIR, tag=MODELS[key],␣
↪project_root=project_root)
    plt.show()

    # Ranking de mas a menos critica (mayor caida primero)
    drops = baseline_accuracy[key] - head_accuracy[key]
    ranking = sorted(
        [(l, h, drops[l, h]) for l in range(n_blocks) for h in range(n_heads)],
        key=lambda t: -t[2],
    )
    print(f"[{key}] Ranking de cabezas por caida de accuracy al apagarlas (mas␣
↪critica primero):")
    for l, h, d in ranking:
        print(f"  block {l+1}, head {h+1}: caida = {d:+.3f}  (accuracy =␣
↪{head_accuracy[key][l, h]:.3f})")
    print()
```

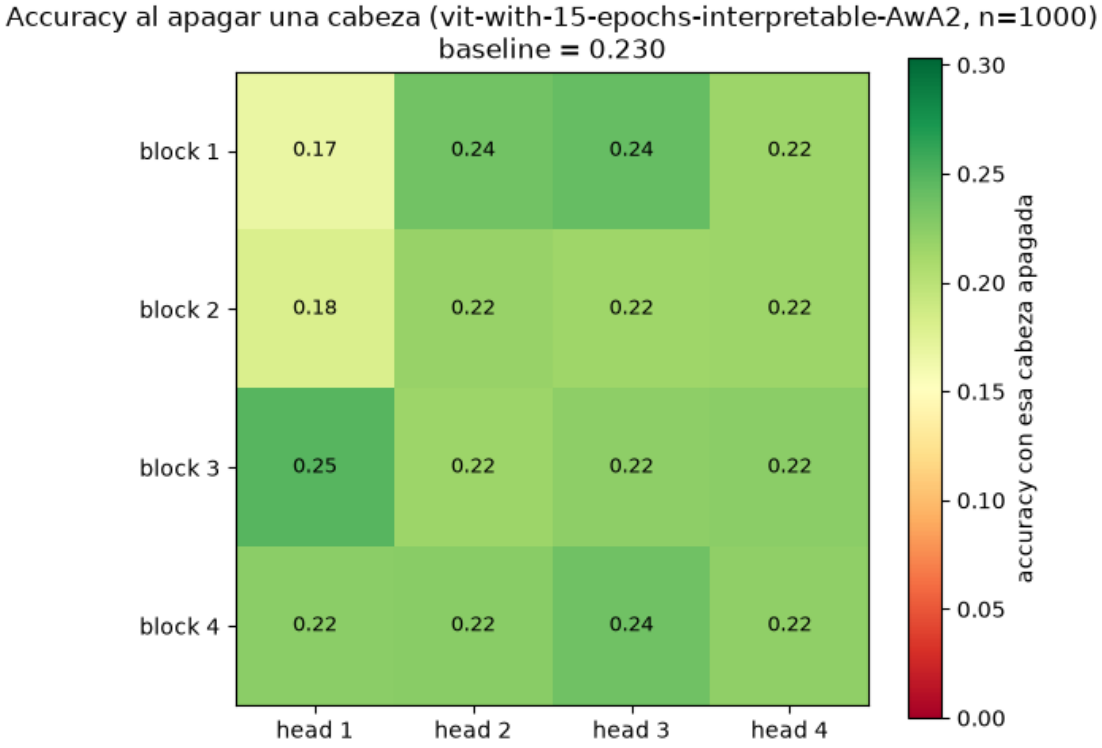
guardado: experimentation/experiments/22082026_ablation/ablation_heads_vit-with-15-epochs-AwA2.png




```
[baseline] Ranking de cabezas por caida de accuracy al apagarlas (mas critica
primero):
  block 2, head 1: caida = +0.054  (accuracy = 0.172)
  block 2, head 3: caida = +0.041  (accuracy = 0.185)
  block 4, head 1: caida = +0.037  (accuracy = 0.189)
  block 4, head 4: caida = +0.031  (accuracy = 0.195)
  block 3, head 2: caida = +0.026  (accuracy = 0.200)
  block 3, head 4: caida = +0.022  (accuracy = 0.204)
  block 4, head 2: caida = +0.015  (accuracy = 0.211)
  block 4, head 3: caida = +0.007  (accuracy = 0.219)
  block 1, head 3: caida = +0.006  (accuracy = 0.220)
  block 2, head 4: caida = +0.006  (accuracy = 0.220)
  block 3, head 1: caida = +0.005  (accuracy = 0.221)
  block 3, head 3: caida = +0.001  (accuracy = 0.225)
  block 1, head 2: caida = +0.000  (accuracy = 0.226)
  block 1, head 4: caida = +0.000  (accuracy = 0.226)
  block 2, head 2: caida = -0.001  (accuracy = 0.227)
  block 1, head 1: caida = -0.007  (accuracy = 0.233)

guardado: experimentation/experiments/22082026_ablation/ablation_heads_vit-
```

with-15-epochs-interpretable-AwA2.png




```
[interpretable] Ranking de cabezas por caida de accuracy al apagarlas (mas
critica primero):
  block 1, head 1: caida = +0.062  (accuracy = 0.168)
  block 2, head 1: caida = +0.049  (accuracy = 0.181)
  block 2, head 3: caida = +0.015  (accuracy = 0.215)
  block 1, head 4: caida = +0.014  (accuracy = 0.216)
  block 2, head 4: caida = +0.014  (accuracy = 0.216)
  block 3, head 2: caida = +0.014  (accuracy = 0.216)
  block 2, head 2: caida = +0.012  (accuracy = 0.218)
  block 4, head 4: caida = +0.008  (accuracy = 0.222)
  block 3, head 3: caida = +0.007  (accuracy = 0.223)
  block 3, head 4: caida = +0.006  (accuracy = 0.224)
  block 4, head 1: caida = +0.006  (accuracy = 0.224)
  block 4, head 2: caida = +0.005  (accuracy = 0.225)
  block 1, head 2: caida = -0.006  (accuracy = 0.236)
  block 4, head 3: caida = -0.007  (accuracy = 0.237)
  block 1, head 3: caida = -0.012  (accuracy = 0.242)
  block 3, head 1: caida = -0.018  (accuracy = 0.248)
```

[18]:
```
head_accuracy['baseline'].mean()
```

[18]:
```
np.float64(0.2108124978840351)
```

[20]:
```
head_accuracy['baseline'].std()
```

[20]:
```
np.float64(0.017353739820738016)
```

[19]:
```
head_accuracy['interpretable'].mean()
```

[19]:
```
np.float64(0.21943750139325857)
```

[21]:
```
head_accuracy['interpretable'].std()
```

[21]:
```
np.float64(0.019653145467300827)
```

1.6 Para reflexionar / siguientes pasos

- ¿Hay cabezas casi prescindibles (accuracy prácticamente igual al baseline) y otras claramente críticas? ¿En qué bloque(s) se concentran las críticas?
- ¿El bloque más crítico a nivel completo (sección 4) coincide con el bloque que tiene la cabeza más crítica (sección 5), o son distintos? (una cabeza puede ser crítica sin que su bloque entero lo sea, si las otras cabezas del mismo bloque compensan parcialmente)
- Con `N_EVAL=500` el ranking ya debería ser bastante estable, pero vale la pena subirlo antes de citar números exactos en la tesis.

La comparación real de B.4 ya está armada arriba: los dos heatmaps guardados (uno por modelo, sección 5) lado a lado — la hipótesis a contrastar es que el penalizado reparte la importancia de forma más uniforme (heatmap más "plano") que el baseline (heatmap con pocas cabezas dominantes). Complemento barato para después: contribución directa al logit por cabeza (proyectar la salida de cada cabeza, ya multiplicada por su parte de `output_projection`, directo al espacio de logits) — más rápido que la ablación porque no requiere volver a correr el forward, pero asume que no hay demasiada interacción no lineal aguas abajo.